

**LAPORAN PRATIKUM
PEMOGRAMAN BERORIENTASI OBJEK**

LAPORAN PRAKTIKUM PEKAN 3

**Disusun Oleh:
Muhammad Aufa Rafiki
2511531012**

**Dosen Pengampu: Nurfiah, S.ST, M.Kom
Asisten Praktikum: Ezza Addini**



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2026

PEMBAHASAN

A. Tahapan Pelaksanaan Praktikum

Langkah-Langkah:

1. Mengunci Entitas Transaksi

Buka file Transaksi.java. Lalu ubah variabelnya menjadi private dan memberikan Getter hanya untuk data yang perlu dibaca. Objek transaksi bersifat immutable atau tidak boleh diubah setelah dibuat, sehingga kita tidak membuat Setter. sistem membatasi akses pada entitas transaksi agar objek bersifat immutable atau tidak dapat diubah setelah dibuat.

```
1 package Praktikum3_2511531012;
2
3 public class Transaksi {
4     private String idTransaksi;
5     private String jenis;
6     private double nominal;
7
8     //konstruktor
9     public Transaksi(String id, String jenis, double nominal) {
10         this.idTransaksi = id;
11         this.jenis = jenis;
12         this.nominal = nominal;
13     }
14
15     public String getIdTransaksi() { return idTransaksi;}
16     public String getJenis() { return jenis;}
17     public double getNominal() { return nominal;}
18
19     public void cetakDetail() {
20         System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
21     }
22 }
23
24 }
```

Penjelasan Kode:

- Atribut Kelas (Private):
Atribut idTransaksi, jenis, dan nominal diubah menjadi private agar tidak bisa diakses atau diubah langsung dari luar *class*.
- Konstruktor
Transaksi(String id, String jenis, double nominal) digunakan untuk mengisi nilai atribut-atribut tersebut saat objek pertama kali diinisialisasi.
- metode Getter (getIdTransaksi(), getJenis(), getNominal()) sehingga data bersifat read-only (hanya bisa dibaca). Tidak ada metode Setter yang dibuat.
- Metode cetakDetail() disediakan untuk menampilkan riwayat transaksi ke layar.

2. Mengunci Entitas Rekening dan Menambahkan PIN

Buka file Rekening.java. Kita akan mengamankan saldo dan menambahkan atribut PIN.

```
1 package Praktikum3_2511531012;
2
3 import java.util.ArrayList;
4
5 public class Rekening {
6
7     private String nomorRekening;
8     private String namaPemilik;
9     private double saldo;
10    private String pin;
11
12    //Implementasi asosiasi (1-to-many)
13    private ArrayList<Transaksi> riwayatTransaksi;
14
15    public Rekening(String nomor, String nama, double saldoAwal, String pinAwal) {
16        this.nomorRekening = nomor;
17        this.namaPemilik = nama;
18        this.saldo = saldoAwal;
19
20        if (pinAwal.length()==6) {
21            this.pin = pinAwal;
22        } else {
23            System.out.println("Peringatan: PIN harus 6 digit! Menggunakan PIN default 123456");
24            this.pin = "123456";
25        }
26
27        //wajib menginisialisasi ArrayList di dalam Konstruktor agar tidak NullPointerException
28        this.riwayatTransaksi = new ArrayList<>();
29
30        System.out.println("Rekening atas nama " + namaPemilik + "berhasil dibuat.");
31    }
32
33    public String getNomorRekening() { return nomorRekening;}
34    public String getNamaPemilik() { return namaPemilik;}
35
36    public boolean otentikasi(String inputPin) {
37        return this.pin.equals(inputPin);
38    }
```

Penjelasan Kode:

- Seluruh atribut (nomorRekening, namaPemilik, saldo, pin, riwayatTransaksi) dikunci dengan modifier private.
- Konstruktor Rekening dimodifikasi untuk menerima parameter pinAwal dan menyertakan validasi logika. Jika panjang pinAwal adalah 6 digit, maka PIN tersebut disimpan. Jika tidak, sistem akan mencetak pesan peringatan dan otomatis menggunakan PIN default "123456".
- Metode Getter: getNomorRekening() dan getNamaPemilik() menyediakan akses read only (hanya baca) untuk mengambil data nomor rekening dan nama pemilik. Metode getPin() dan setSaldo() sengaja tidak dibuat untuk mencegah kebocoran kode rahasia serta manipulasi nilai saldo secara tidak sah dari luar kelas.
- Sistem menggunakan metode otentikasi(String inputPin) untuk

mencocokkan input PIN dari pengguna dengan atribut pin yang tersimpan. Metode ini akan mengembalikan nilai true jika cocok, dan false jika salah.

B. Latihan/Tugas

1. Menu 1 (Buka Rekening): Sebelum menginstansiasi objek baru, minta user menginputkan PIN (String angka 6 digit).

```
case 1:
System.out.print("Masukkan No Rekening: ");
String no = input.nextLine();
boolean sudahAda = false;
for (Rekening r : daftarRekening) {
    if (r.getNomorRekening().equalsIgnoreCase(no)) {
        sudahAda = true;
        break;
    }
}
if (sudahAda) {
    System.out.println("Error: Nomor rekening tersebut sudah terdaftar!");
} else {
    System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukkan Saldo Awal: ");
    Double saldo = input.nextDouble();
    input.nextLine();
    //Input PIN
    System.out.print("Masukkan PIN (6 digit): ");
    String pin = input.nextLine();

    Rekening rekeningBaru = new Rekening(no, nama, saldo, pin);
    daftarRekening.add(rekeningBaru);
    akunAktif = rekeningBaru;
}
break;
```

Penjelasan Kode:

- Input Nomor Rekening: Program meminta pengguna memasukkan nomor rekening yang diinginkan.
- Pencegahan Duplikasi: Program melakukan perulangan (looping) untuk mengecek seluruh daftar rekening yang tersimpan. Penggunaan `r.getNomorRekening()` menerapkan prinsip enkapsulasi (getter). Jika nomor rekening yang diinput sudah ada di sistem, variabel `sudahAda` diubah menjadi true.
- Jika `sudahAda` bernilai true: Tampil pesan error bahwa nomor rekening tidak bisa digunakan karena sudah terdaftar.
- Jika `sudahAda` bernilai false: Program berlanjut ke blok else untuk

memproses pembuatan rekening baru.

- Program meminta data Nama Pemilik dan Saldo Awal.
- `input.nextLine()` setelah `nextDouble()` berfungsi untuk membersihkan karakter enter (buffer) agar input berikutnya berjalan lancar.
- Program meminta pengguna memasukkan PIN 6 digit yang akan digunakan sebagai pengaman transaksi.
- Pembuatan Objek: Membuat objek rekeningBaru dari kelas Rekening dengan menyertakan parameter no, nama, saldo, dan pin ke dalam konstruktor.
- Objek baru disimpan ke dalam list (`daftarRekening.add`).
- Objek yang baru dibuat secara otomatis dijadikan sebagai akun yang sedang diakses saat ini (`akunAktif`).

Output:

```
==== SISTEM PERBANKAN MINI ====
[ Akun Aktif: Belum ada akun terpilih ]

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Ganti PIN
0. Keluar
Pilih menu:1
Masukkan No Rekening: 2511531012
Masukkan Nama Pemilik: aufa
Masukkan Saldo Awal: 200000
Masukkan PIN (6 digit): 123789
Rekening atas nama aufa berhasil dibuat.

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Ganti PIN
0. Keluar
Pilih menu:1
Masukkan No Rekening: 1012
Masukkan Nama Pemilik: dafa
Masukkan Saldo Awal: 99999999
Masukkan PIN (6 digit): 1234567
Peringatan: PIN harus 6 digit! Menggunakan PIN default 123456
Rekening atas nama dafa berhasil dibuat.
```

Penjelasan Output:

Jika panjang pin adalah 6 digit, maka PIN tersebut disimpan. Jika tidak, sistem akan mencetak pesan peringatan dan otomatis menggunakan PIN default "123456".

2. Keamanan Transaksi: Perbarui Menu 3 (Tarik Tunai) dan Menu 6 (Cetak Mutasi). Sebelum sistem mengeksekusi kedua method tersebut, sistem harus meminta user memasukkan PIN.

- Panggil fungsi `akunAktif.otentikasi(pinYangDiinput)`.
- Jika mengembalikan nilai `true`, jalankan transaksi.
- Jika `false`, batalkan dan cetak: "Akses Ditolak: PIN yang Anda masukkan salah!".

Jawaban:

- Perbarui Menu 3 (Tarik Tunai)

```
case 3:
    if (akunAktif == null) {
        System.out.println("Error: mohon maaf, anda belum memiliki nomor rekening!");
    } else {
        // Minta PIN sebelum Tarik Tunai
        System.out.print("Masukkan PIN (6 digit): ");
        String inputPin = input.nextLine();

        // Otentikasi PIN
        if (akunAktif.otentikasi(inputPin)) {
            System.out.print("Masukkan nominal tarik tunai: ");
            double tarik = input.nextDouble();
            input.nextLine();
            akunAktif.tarikTunai(tarik);
        } else {
            System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
        }
    }
    break;
```

Penjelasan Kode:

- Program mengecek apakah variabel `akunAktif` bernilai `null`.
- Jika `null`, berarti pengguna belum membuka atau memilih rekening. Program mencetak pesan error dan proses penarikan dibatalkan.
- Jika akun aktif tersedia, program meminta pengguna menginputkan PIN terlebih dahulu sebagai verifikasi keamanan sebelum penarikan uang diproses.

- Pemanggilan `akunAktif.otentikasi(inputPin)` mengirimkan PIN input untuk dicocokkan dengan PIN rahasia yang tersimpan di dalam objek Rekening.
- Jika Otentikasi Berhasil (`true`): Program meminta masukan nominal uang (tarik), memanggil `input.nextLine()` untuk membersihkan karakter enter (buffer), lalu memanggil method `akunAktif.tarikTunai(tarik)` untuk memproses pengurangan saldo.
- Jika Otentikasi Gagal (`false`): Transaksi dibatalkan secara otomatis dan sistem menampilkan pesan "Akses Ditolak: PIN yang Anda masukkan salah!".

Output:

Jika pin salah maka muncul pesan akses ditolak

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Ganti PIN
0. Keluar
Pilih menu:3
Masukkan PIN (6 digit): 123456
Akses Ditolak: PIN yang Anda masukkan salah!
```

Jika pin benar maka tarik tunai dapat dilakukan

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Ganti PIN
0. Keluar
Pilih menu:3
Masukkan PIN (6 digit): 123789
Masukkan nominal tarik tunai: 12000
Tarik tunai Rp12000.0 berhasil. Saldo saat ini: Rp188000.0
```

- Menu 6 (Cetak Mutasi).

```
case 6:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        // Minta PIN sebelum Cetak Mutasi
        System.out.print("Masukkan PIN (6 digit): ");
        String inputPin = input.nextLine();

        // Otentikasi PIN
        if (akunAktif.otentikasi(inputPin)) {
            akunAktif.cetakMutasi();
        } else {
            System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
        }
    }
    break;
```

Penjelasan Kode:

- Program mengecek apakah variabel akunAktif bernilai null.
- Jika null, berarti belum ada akun yang dibuat atau dipilih. Program akan mencetak pesan error dan proses pencetakan mutasi langsung dibatalkan.
- Jika akun aktif tersedia, program meminta pengguna menginputkan PIN terlebih dahulu. Hal ini dilakukan untuk menjaga privasi dan keamanan data riwayat transaksi nasabah.
- Memanggil fungsi akunAktif.otentikasi(inputPin) untuk mencocokkan input pengguna dengan PIN yang tersimpan di dalam objek Rekening tanpa mengakses variabel pin secara langsung.
- Jika Otentikasi Berhasil (true): Sistem mengeksekusi metode akunAktif.cetakMutasi() untuk menampilkan seluruh riwayat transaksi.
- Jika Otentikasi Gagal (false): Akses ditolak dan sistem mencetak pesan "Akses Ditolak: PIN yang Anda masukkan salah!".

Output:

Jika pin salah

```
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cetak Mutasi (Riwayat)  
7. Ganti PIN  
0. Keluar  
Pilih menu:6  
Masukkan PIN (6 digit): 123456  
Akses Ditolak: PIN yang Anda masukkan salah!
```

Jika pin benar maka riwayat dapat ditampilkan

```
[ Akun Aktif: aufa ( No Rekening: 2511531012) ]  
  
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cetak Mutasi (Riwayat)  
7. Ganti PIN  
0. Keluar  
Pilih menu:6  
Masukkan PIN (6 digit): 123789  
  
--- Riwayat Transaksi ---  
ID: TRX-T-1790523194384 | Jenis: Debit | Nominal: Rp12000.0
```